

Quiz - Aulas 1 a 3

10 questões - versão 42

Nome: _____ Data: ____/____/____

1. Numa linguagem de programação, o que pertence ao nível LÉXICO? [Três níveis de uma linguagem]

- a) a velocidade do processador
- b) o que é um condicional
- c) palavras reservadas (if, for)
- d) a gramática da linguagem

2. Qual regex casa "bnana" e "banana", mas NÃO casa "bxnana"? [Expressões regulares]

- a) `ba+nana`
- b) `b.nana`
- c) `ba*nana`
- d) `ban+ana`

3. Qual é o PRIMEIRO trecho que a regex `ba*nana` casa no texto abaixo? [Expressões regulares]

Comprei bnana, banana e baaanana no mercado.

- a) `bnana`
- b) `baaanana`
- c) `bna`
- d) `banana`

4. Depois de executar o programa abaixo, o que `.s` mostra? (topo à direita) [Forth]

```
7 6 SWAP OVER
```

- a) `<4> 7 6 7 6`
- b) `<3> 6 7 6`
- c) `<3> 7 6 7`
- d) `<3> 6 7 7`

5. O que o REPL do Racket mostra ao avaliar a expressão abaixo? [Lisp / Racket]

```
(cond
  [(< -3 0) "negativo"]
  [(= -3 0) "zero"]
  [else "positivo"])
```

- a) `"negativo"`
- b) `"positivo"`
- c) `"zero"`
- d) `erro: cond: no matching clause`

6. Qual das expressões abaixo dá ERRO no Racket? [Lisp / Racket]

- a) `(cdr '(5))`
- b) `(eval '(* 2 3))`
- c) `(/ 5)`
- d) `(cdr 5)`

7. Qual das expressões abaixo produz o valor '(9) ? [Lisp / Racket]

- a) `(cdr '(4 3 9))`
- b) `(cdr (cdr '(4 3 9)))`
- c) `(car (cdr (cdr '(4 3 9))))`
- d) `(cdr (cdr '(3 9)))`

8. Pelo "pulo do gato", ao escrever dobrar o que você assume sobre a chamada recursiva (dobrar (cdr l)) ? [Recursão sobre listas]

- a) que precisa ser simulada passo a passo, chamada por chamada, antes de escrever a função
- b) que devolve a cabeça da lista
- c) que só pode ser usada depois que a lista inteira for percorrida com um laço
- d) que já é a cauda dobrada, pronta; falta só encaixar a cabeça dobrada na frente com cons

9. A definição de my-filter abaixo está errada. Qual é o erro? [Recursão sobre listas]

```
(define (my-filter p l)
  (if (null? l)
      '()
      (if (p (car l))
          (list (car l) (my-filter p (cdr l)))
          (my-filter p (cdr l)))))
```

- a) A chamada recursiva é sobre l, não sobre (cdr l): a lista nunca diminui.
- b) Falta o teste (null? l): a recursão não tem onde parar.
- c) O caso base tem o valor errado para o tipo de retorno.
- d) Usa list em vez de cons: aninha a cauda em vez de encadear.

10. Qual das definições de my-map está correta? ((my-map add1 '(1 2 3)): '(2 3 4)) [Recursão sobre listas]

a)

```
(define (my-map f l)
  (if (null? l)
      '()
      (cons (f (car l)) (my-map f (cdr l)))))
```

b)

```
(define (my-map f l)
  (if (null? l)
      '()
      (my-map f (cdr l))))
```

c)

```
(define (my-map f l)
  (cons (f (car l)) (my-map f (cdr l))))
```

d)

```
(define (my-map f l)
  (if (null? l)
      '()
      (cons (car l) (my-map f (cdr l)))))
```